

第 2 册 Agent 工作流与记忆管理

本册完整收录对应专题的原始内容，保留代码事实、现有限制与改进建议的区分。供开发维护和 AI 项目理解使用，不应直接发布为面向客户的售后政策。

本册专题：聊天链路与 Agent 工作流；记忆与会话恢复

聊天链路与 Agent 工作流

一条客户消息如何完成

客户页面把输入绑定到 Pinia 的 draft。`useAgentChat.submitDraft` 先检查是否正在发送、是否正在恢复历史以及企业是否选定，再取出非空输入，在页面追加乐观显示的客户消息，并创建 `client_request_id`。输入框此时清空，不代表消息已持久化。

`agent-service.ts` 经统一 Axios 客户端提交到 `POST /api/v1/agent/chat`。后端 Pydantic 验证消息、企业、可选会话 ID 和请求键，Cookie 认证解析真实客户。客户端不提交可信 `customer_id`，也不能提交自己的 `thread_id`。

`ChatRequestService.chat` 首先检查企业与会话归属，再尝试占有持久化请求键。重复请求可能直接返回旧响应，不会重新运行 Graph。首次请求进入 `ConversationService.chat`；已有会话先获得跨进程会话锁，然后保存当前客户消息。如果已经人工接管，本轮到这里按人工模式返回，不启动 Agent。

AI 模式下，服务先创建 running 运行记录，读取业务消息，再调用 `run_graph_turn`。该函数构造可信 `thread_id`、检查是否有未完成检查点、首次导入历史或只追加当前 `HumanMessage`，然后执行图。Graph 完成后，服务保存客户可见的 AI 回复、人工排队状态、工具摘要和运行结果，最后请求层保存可重放响应。

浏览器校验响应结构后显示最终回复。工单、修改或查询提示来自结构化字段，而不是搜索回复中有没有“已创建”。页面会刷新历史、接管状态和相关工单缓存。网络失败则走专门的结果核对提示，不直接把它当作没有执行。

图和模型的装配

`factory.get_support_graph` 延迟创建并缓存 Graph。记忆资源必须已由 FastAPI 生命周期初始化；缺少聊天密钥时抛出配置错误。延迟创建使 `/health` 正常不能证明模型配置正常。

当前客户端使用配置的模型名，代码默认 `deepseek-v4-flash`，`temperature` 为 0，单次调用超时 30 秒，SDK 自动重试为 0。请求体显式带关闭 `thinking` 的供应商参数。这是当前适配代码，不是对所有供应商支持性的通用保证；切换供应商需要重新验收协议。

同一个客户端派生多种接口：`AgentDecision` 的结构化意图输出；强制 `create_support_ticket` 的普通建单调用；仅绑定 `query_support_tickets` 的只读工单查询；绑定 `query_my_orders` 与 `create_order_ticket` 的订单调用；`KnowledgeAnswer` 与 `AfterSalesAssessment` 的结构化结果。它们不是多个独立部署或单独训练的模型。

`method="function_calling"` 让兼容模型按函数参数 Schema 生成可解析结构。用于结构化判断时，函数调用是输出约束；它本身不会自动执行数据库工具。业务工具必须由 Graph 允许，再交给 ToolNode。

所有这些模型接口外包 `WindowedModel`，每次真正调用前检查并裁剪输入。`cast` 是 Python 静态类型提示，不执行类型转换，也不验证服务端模型能力。

图入口与状态清理

真实入口是 START 到 `load_customer_memory_node`，再到 `analyze_request_node`。加载节点读取当前企业、客户的显式回复偏好，并清除上一轮的工单结果、工具名、最终回复、RAG 命中与预判检查标志。否则旧检查点中的工单编号可能被错误当成这一轮又建单成功。

意图节点把系统提示词、结构化订单/修改草稿和会话视图传给判断模型。`decision_history` 不修改原检查点，而是为分类模型移除历史 AI 工具调用指令，把 ToolMessage 改成只读历史说明，降低分类模型照着旧协议再次调用工具的可能性。

明确的转人工口令、活动流程中的数字序号、部分短诉求、取消以及确认修改/确认提交，有程序直接识别的分支。这些分支降低短消息被模型误解的概率，但不是完整中文语法解析器。其他表达仍依赖结构化模型判断。

AgentDecision 要求意图、优先级、是否需要人工、是否建单、是否补信息、原因和回复等字段。Pydantic 拒绝未知字段和互相冲突的主要分支，例如纯知识问答不能同时创建工单，修改工单不能同时查询订单。普通建单就绪时必须要有标题和描述。

结构化格式失败会有限纠正，最多两次尝试，只提示错误字段和类型，不把原始异常或客户隐私整段塞回纠错提示词。纠正失败才报 `model_output_invalid`；格式纠正不是工具写入重试。

节点和分支的真实职责

`automatic_reply_node` 使用意图判断生成的 `reply_draft`，结束普通咨询。它不执行检索，因此企业政策事实应由意图节点正确路由到知识分支；分类准确率依然需要真实模型评测。

`collect_ticket_details_node` 只发送针对缺失内容的追问，结束本轮，不循环等待用户。客户下一次发送消息后从 START 开始新一轮，用检查点中的状态接续。

`human_handoff_node` 生成确定性的排队说明。实际排队字段由会话服务与最终回复一起提交。节点不会声称已经有人接听。普通明确建单场景可以先建单并标记需要人工，不能把 `requires_human` 在所有路径中都解释为“禁止登记工单”。售后预判无法确认时则停止建单并交人工。

`request_ticket_creation_node` 让专用模型产生一个 `create_support_ticket` 调用。服务端要求恰好一个指定工具，`title`、`description`、`category`、`priority` 与已验证值完全一致；最多纠正一次。通过后进入 `ticket_tools`，成功后到 `finalize_ticket_node`。

`finalize_ticket_node` 使用工具实际回写的工单编号生成成功话术，没有编号则报错，不允许在工具之前提前承诺。关联订单工单额外说明没有执行实际退款。它生成的是模板回复，不要求模型再决定一次是否需要写入。

`ticket_query_agent_node` 与 `query_tools` 构成只读循环。首次必须查询数据库，不能凭历史直接报告实时状态；每次只允许一个 `query_support_tickets`。结果不足可以缩小条件，一轮最多三次查询。查询有整

体时间预算，工具自身也受预算限制，错误不会冒充零条结果。最终文本结束本轮。

`prepare_ticket_edit_node` 处理候选展示、选择、最新快照、差异预览和确认。准备阶段直接调用只读 `Service` 获取数据，不是每一次只读查询都对应独立 Tool Call。确认写入通过 `edit_tools` 中的 `update_support_ticket`；`finalize_ticket_edit_node` 使用持久化结果形成回复。

`order_agent_node` 是模型驱动的受限订单循环，用于查询等兼容路径；`order_workflow_node` 是程序驱动的多轮订单建单节点，用结构化记忆稳定保存用户选择。两者都可以进入 `order_tools`，工具后再根据状态返回对应节点。真实实现不是只有一个完全自由的订单 Agent。

`prepare_knowledge_node` 构造 `search_company_knowledge` 调用，`knowledge_tools` 执行检索，`knowledge_answer_node` 校验来源并回答，然后结束。这个独立分支只挂只读检索工具。

`assess_after_sales_node` 位于订单 workflows 内部：订单查询得到唯一记录后检索政策，工具结束路由到预判节点。预判需要追问、确认或人工处理时结束；只有本轮确认有效且快照一致时回到订单 workflows 执行建单。

工具的权限边界

当前主要业务工具共六个：`create_support_ticket`、`query_support_tickets`、`query_my_orders`、`create_order_ticket`、`update_support_ticket`、`search_company_knowledge`。旧 `append_ticket_comment` 不在当前图的可执行工具列表中，但旧结果协议和记录兼容仍在。

ToolRuntime 注入 `SupportToolContext`，包括数据库 Session、会话 ID、企业、客户、请求操作 ID 和记忆代次。这些不是模型可自由填写的参数。工具通过 `Service` 和 `Repository` 再次核对数据范围。

工具返回 LangGraph Command，同时更新明确的 State 字段和与调用 ID 对应的 ToolMessage。工具调用消息、工具结果消息和业务最终回复各有作用，不能拿一段“我查询了”的自然语言替代它们。

ToolNode 配置 `handle_tool_errors=False`，未在业务内部处理的异常向上传播，由请求服务核对副作用并返回安全错误。知识检索有特定的可恢复错误，返回 `knowledge_error` 供节点保守回答；这不等于吞掉所有数据库或权限错误。

普通订单建单的短期状态

OrderMemory 保存 `candidates`、`selected`、`reference`、`issue` 和 `stage`。候选保存展示顺序，因此“1”“2”能映射上一轮看见的记录。无效序号不沿用旧选单；多笔结果不默认第一单。

每轮仍重新查真实订单，当前 `selected` 只是上下文，不直接作为数据库写权限。最终 `create_order_ticket` 要求本轮可用 ID 列表恰好等于一个目标订单，且没有已经创建的工单、存在创建授权。创建后记忆变为 `completed`，下一轮不能重复沿用确认。

普通建单最小诉求长度为两个字符，最长 2000。工单标题取诉求前 200 字符。单独“退款”会被内容处理逻辑放入售后诉求，问题描述可能为空，这是和普通人工建单描述最短两字符不同的内部订单路径，不应使用同一表单假设反推所有数据库行。

售后预判的状态和授权

预判在 OrderMemory 中增加 `assess`、`facts`、`reported_received_on`、`assessment_fingerprint`、`assessment_reply`、`confirmed` 和 `clarification_rounds`。状态新增 `collect_conditions`、`confirm`。补充事实有顺序，最多保留 4000 字符，提示模型以最新更正为准；这不是经过企业验证的真相库。

预判只使用本轮检索结果中 `source_kind` 为 `document` 的正式资料。没有正式资料、向量服务不可用、日期未来、商品或诉求超范围、模型超时、解析错误、无有效引用等情况会进入人工核实，不建单。真实风险还要依赖提示词识别，系统并非完备安全规则引擎。

签收日期先采用订单记录，缺少时用客户明确自述日期。仍未知就追问，不用创建时间代替。追问计数超过两轮交人工，避免无限循环。结构化结果分 `eligible`、`needs_info`、`manual`；`eligible` 只表示可提交客服审核，不代表权益已经获批。

预览记录指纹，包含选定订单快照、诉求、自述事实、自述签收日、北京时间当天和本轮正式命中片段的 ID 与内容。客户只能用当前明确文本“确认提交”授权，不接受模型把“可以吗”“知道了”推断为提交。

确认时重新查订单和政策，指纹相同才保留确认，变化就重新展示结论并请客户再次确认。这个机制检查的是实际命中快照，不是整个企业知识库所有文档的全局版本；没有命中的政策修改不会必然改变指纹。读取政策到建单之间也不是跨全部表的全局锁定事务，不能宣传成绝对无时间竞态的审批系统。

预判生成的结论仍依赖 LLM 理解政策，来源序号校验只证明引用存在，不证明推理必然正确。程序可以控制何时写入、写到哪张单，却不能仅凭 JSON 结构保证所有资格判断正确。因此真实使用需要人工验收政策解释，并保持最终人工审核语义。

状态结束与用户取消

追问和确认预览通常是正常到 END，不是 LangGraph interrupt 暂停。在下一条用户消息到来时用同一 `thread_id` 开启新一轮次，读取之前记忆。不能因为流程要等用户就说项目已经使用 `interrupt/resume` 人工审批接口。

客户取消订单准备会清除这次草稿授权，不删除既有工单；取消修改同样只撤销未提交的编辑。人工接管恢复 AI 时会切换记忆代次。已经提交的写入不能通过一句“取消”自动回滚。

本篇的主要定位文件是 `backend/app/agent/graph.py`、`factory.py`、`tools.py`、`schemas.py`、`state.py`、`decision_context.py` 和 `workflows/` 下四个流程文件。

记忆与会话恢复

先区分四种保存

业务消息保存在 PostgreSQL 的 `conversations/messages`，供页面展示、历史查询和审计。短期流程状态保存在官方 LangGraph Checkpointer，包括消息链、候选、选择、编辑草稿和节点执行状态。长期偏好保存在官方 Store，供同一客户在同一企业的不同会话使用。尚未发送的客户输入保存在浏览器 Pinia 内存中，不是前三者的一部分。

所以“会话不会丢”必须明确对象：已经提交的业务消息通常能恢复，正常完成轮次的流程草稿通常能接续，但未发送的输入不会跨刷新自动保存，执行中断也不是任意无脑续跑。

短期记忆的官方实现

`agent/memory/persistence.py` 使用 `AsyncPostgresSaver`，不是自建一个 Python 字典模拟持久化。连接池由 FastAPI lifespan 管理，最少一个、最多八个连接，使用 `autocommit`、`dict_row` 和禁用预编译阈值的连接参数。初始化通过官方 setup，多个进程用数据库 advisory lock 串行化初始化。

`services/agent_memory_service.py` 中的 `conversation_config` 创建 `thread_id`，基本形式是企业 UUID、客户 UUID、会话 UUID 用冒号连接。恢复人工后的新代次追加 `:resume:代次`。这个值由认证和数据库上下文生成，不接受浏览器指定。

每次调用先读取检查点。没有检查点才导入业务历史一次；已有检查点仅追加当前 `HumanMessage`。消息 ID 使用业务消息 UUID，配合 `add_messages` 进行合并，不反复把整个数据库历史拼进去。人工回复在首次导入图历史时映射为 `AIMessage`，用来恢复对话背景，但业务表仍保留 `staff` 角色。

图使用 `durability="sync"` 写检查点。同步检查点不意味着它与工单事务属于同一次原子提交，发生进程或数据库故障时仍必须核对业务回执。

`JsonPlusSerializer` 限定可恢复业务类型，包含订单记忆、工单修改记忆、相关结果和枚举，保留旧模块路径白名单。没有打开任意 pickle 反序列化。`agent/order_memory.py`、`agent/comment_memory.py` 是旧检查点兼容导入入口，不能因为主逻辑搬到 `memory` 文件夹就直接删除。

长期记忆的具体范围

`AsyncPostgresStore` 的命名空间由 `supportdesk`、企业 ID、客户 ID、preferences 组成，固定键 `reply` 保存偏好。当前字段主要是 `reply_style`，支持 `concise`、`detailed`。保存覆盖这个稳定键，不把每一条聊天重复累积成永久事实。

长期偏好使用 `index=False`，不是向量语义记忆。它不会自动从“我姓王”提取姓名并永久保存，不保存用户的所有关系，不代替客户账号，也不复制订单或工单。

客户可以通过当前企业下 preferences 的 GET、PUT、DELETE 接口读、写、清除，前端尚无专门设置页面。偏好影响模型组织语言，不修改权限、退款规则、明确确认要求和确定性模板。跨企业不共享这份偏好。

消息裁剪不等于删除历史

`context_window.py` 在调用模型前使用官方 `trim_messages`，默认非系统历史最多 40 条，Token 总预算默认 12000，其中预留 2048，字符与 Token 比例估算默认 2.0。前置 `SystemMessage` 也占 Token 预算，但不被当成普通历史随意删掉。

算法保留前置系统指令和最近完整用户轮次，先按消息条数，再按估算 Token 裁剪；不会只截取客户半句话或截断工具 JSON。最终历史必须从 `HumanMessage` 开始，`AI Tool Call` 与 `ToolMessage` 必须匹配，调用 ID 不能缺失、重复或孤立。

如果当前完整轮次连同系统提示已超预算，系统会报上下文错误，而不是悄悄把最关键内容截断。该算法估计输入体积，不是供应商精确分词计费，也不能保证所有额外协议开销都被精确覆盖。

裁剪返回新列表给模型，不回写删除 `Checkpoint`，不删除 `messages` 表。旧历史仍在数据库，但可能不再出现在最近模型窗口，所以不能承诺模型永远记得很久前的任意一句话。结构化订单/修改状态能保留关键流程信息，长期偏好只保留显式设置项。

误退出和恢复场景

用户操作	已发送内容与流程	未发送内容与限制
在同一标签页离开聊天，进入工单或订单，再返回同一家企业	Pinia 通常保留当前会话 ID；服务端历史可重新读取	客户草稿在应用未重载时保留
切换到另一企业	当前会话选择被清除，新企业要独立选择历史；服务端原会话仍在	按企业存入内存 Map，切回可取回对应草稿
刷新浏览器页面	数据库内容不删除，但当前会话不会自动从 localStorage 认领；需打开历史会话选择	Pinia 重建，草稿和页面内待重试键丢失
关闭标签页或浏览器再打开	若 Cookie 仍有效可以继续登录状态；否则重新登录，从企业历史选会话	未发送草稿没有磁盘持久化，不能保证恢复
退出客户账号	服务端撤销当前客户登录会话，历史消息、工单、正常检查点和偏好不删除	清空 Query 缓存、当前会话和全部客户草稿
改用另一客户账号	读取的是新客户授权范围，不能继承旧客户数据	身份变化会重置草稿，防止泄露给下一位使用者
点击新建会话	清除前端当前会话选择，旧数据库会话仍可从历史查找；首条新消息才真正创建新会话	clearConversation 本身不清空 draft，现有输入可能留在新会话中，发送前应核对
选择另一条历史会话	用所选 ID 读取服务端历史，不能靠前端旧消息冒充	草稿按企业而不是按会话隔离，同企业切换历史时应检查输入是否属于当前会话
后端正常重启	正常完成的数据库与检查点数据仍在	浏览器草稿取决于浏览器是否重载，不由后端保证
关闭或停止 Docker 容器	数据卷保留时重启仍有数据，停机期间无法服务	删除数据卷是另一种破坏性操作，不在恢复保证内
发送后立即离开或断网	服务端可能继续执行，也可能尚未完成，必须核对历史和回执	前端的乐观气泡不证明数据库已提交

localStorage 的真实用途

stores/conversation.ts 仍在 setConversationId 时写一个会话 ID，并在 clearConversation 时删除，但初始化明确从 null 开始，不自动读取并认领它。仅看见 localStorage 写入代码，不能得出“刷新自动恢复最后一段会话”的结论。

客户草稿在内存 Map 按企业存放，没有 sessionStorage、IndexedDB 或数据库草稿表。这个选择减少敏感未提交输入长期留在共享设备中的风险，但也牺牲刷新恢复能力。当前没有离开客户聊天页面前的通用未保存警告。

人工回复输入框是组件局部状态，组件被关闭、重建或刷新时也没有可靠自动恢复。知识库分块保存到服务器后才能跨刷新恢复；编辑器关闭抽屉时有未保存确认，但浏览器刷新、整个路由离开不等于都被同一确认拦截。

恢复历史的具体操作

客户重新登录原账号，选择原企业，进入聊天页，展开“查看与该企业的历史会话”，按日期和会话 ID 前缀选择原会话。列表默认每页二十条，按 `updated_at` 倒序查询。业务消息接口按创建时间升序返回消息，和“历史会话列表最近优先”是不同排序。

当前消息列表恢复采用整体替换，发送期间暂不使用轮询快照覆盖乐观消息。发送结束后以数据库历史为准，因此先前仅在页面显示、未被服务器保存的临时气泡可能消失，这不等于服务器删除了历史。

订单或编辑流程中正常返回的追问，会伴随检查点保存。恢复同一会话继续回答，可以利用上次候选和草稿。新建会话则有新的 `thread_id`，不应该默认继承旧会话的选单授权；跨会话只共享同企业、同客户的长期回复偏好。

未完成检查点的恢复边界

如果检查点存在后续待执行节点，系统默认不把新消息当作恢复旧执行的授权。因为旧工具可能已经写入数据库，再执行可能重复建单。这时返回 `checkpoint_recovery_required`，要求先查工单和运行记录。

只有一个明确可自动关闭的例外：待执行节点恰好是 `analyze_request_node`，存在确定错误、没有 `interrupt`，并且没有进入任何工具阶段。服务使用官方 `update_state` 将这次前置失败收尾，下一条消息重新从入口运行，不用 `ainvoke(None)` 重播旧工具。

其他中断没有通用自动恢复按钮，不应该直接删检查点、换请求键或新建会话来掩盖结果未知。新建会话可以隔离对话状态，但不能证明上一次业务副作用没有发生。

当前没有自动历史摘要、检查点定期清理、业务历史归档和用户一键删除全部记忆的完整产品流程。删除长期偏好只删除 `reply` 偏好，不等于删除聊天或工单。